

Tarot en C++

Table des matières

I	Présentation du projet	2
1	Introduction	2
2	Règle de jeu	2
2.1	La distribution des cartes	2
2.2	Les enchères	2
2.3	Le chien	2
2.4	Le jeu	3
2.5	Le comptage des points	3
II	Représentation UML	4
III	Mise en oeuvre	5
3	Pour commencer	5
4	Card	5
5	Deck	5
6	Player	6
6.1	Classe abstraite	6
6.2	HumanPlayer	7
6.3	IAPlayer	7
7	Game	8
IV	Conclusion	9

Première partie

Présentation du projet

1 Introduction

En ce premier semestre de Licence 3, il nous a été demandé de créer un tarot en C++.

Le but de cet exercice, ou projet si l'on préfère, est de nous enseigner par la pratique l'utilisation des classes dans ce langage. Il nous faudra donc, dans un premier temps, avoir une bonne conception du jeu, et départager les actions de ce dernier en différents objets.

2 Règle de jeu

Il semble approprié de rappeler quelques règles de jeu du tarot. Prenons le cas où il y a 4 joueurs, un paquet est composé de 78 cartes : 4 couleurs de 14 cartes chacune avec ordre décroissant (Roi, Dame, Cavalier, Valet, 10, 9, 8, 7, 6, 5, 4, 3, 2, As), et 21 atouts plus l'excuse. Le 1, le 21 et l'excuse sont appelés bouts, ou oudlers.

2.1 La distribution des cartes

- la premier joueur distribué est celui à droite du donneur, ce dernier évoluant au cours du jeu,
- chaque joueur reçoit 18 cartes, le donneur les distribuant 3 par 3, et mettant une carte dans le chien à chaque tour.

2.2 Les enchères

- chaque joueur annonce un contrat, parmi la prise, la garde, la garde sans et la garde contre, en commençant par le premier joueur distribué,
- le joueur ayant pris le plus gros contrat devient le preneur de la manche, en cas d'égalité avec un autre, c'est le premier à avoir annoncé qui l'emporte.

2.3 Le chien

- en cas de prise ou de garde, le preneur peut échanger des cartes de sa main avec celle du chien, il met ensuite le chien dans son tas.
- en cas de garde sans, les cartes du chien vont directement dans le tas du preneur,

- en cas de garde contre, ces dernières sont données aux autres joueurs, appelés défenseurs.

2.4 Le jeu

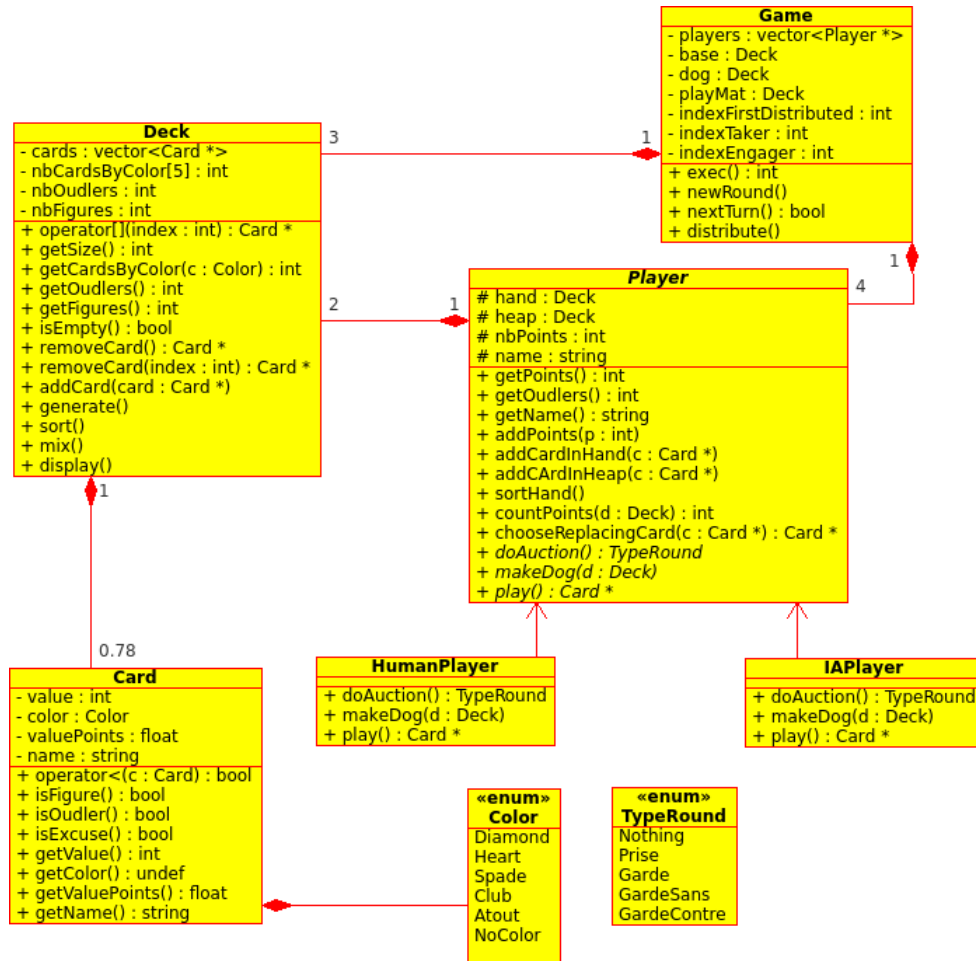
- il se déroule au tour par tour,
- chaque joueur pose une carte à tour de rôle,
- celui qui remporte un pli est le premier à jouer au tour suivant,
- le premier joueur peut jouer la carte de son choix, les autres doivent se conformer à la couleur posée, ou mettre de l'atout si la couleur jouée leur fait défaut, et sinon mettre n'importe quelle couleur mais ils perdront forcément le pli.

2.5 Le comptage des points

- chaque atout et carte basse vaut 0.5 points,
- les bouts valent tous 4.5 points,
- le roi vaut 4.5 points, la reine 3.5, le cavalier 2.5, le valet 1.5,
- Le preneur s'est engagé à réaliser un minimum de points selon le nombre de bouts qu'il possède dans son tas à la fin de la partie : 3 bouts - 36 points, 2 bouts - 41 points, 1 bout - 51 points, aucun bout - 56 points.
- s'il le réalise, le nombre de points excédentaire est ajouté à 25, sinon le nombre de points manquants est soustrait.
- ensuite, ce total est assorti à un multiplicateur variant selon le contrat : x1 prise, x2 garde, x4 garde sans, x6 garde contre.
- si le contrat a été réalisé le preneur gagne 3 fois ce total, les défenseurs le perdent 1 fois, si cela n'a pas été le cas, le preneur perd 3 fois ce total et les défenseurs le gagnent 1 fois.

Deuxième partie

Représentation UML



Troisième partie

Mise en oeuvre

3 Pour commencer

Nous voyons bien ici que ce projet demande des classes facilement déterminables, si l'on s'y attarde peu de temps.

Je vais donc commencer par la classe la plus simple, et la plus 'basique', la classe concernant une carte (Card).

4 Card

Description Une carte à une couleur, une valeur, un nombre de points associés, et un nom. Ce dernier sera déterminé automatiquement dans le constructeur. Une exception est générée si la carte ne peut être déterminée, car non-existante.

Nous pouvons aussi savoir si une carte est un bout (oudler), une tête (figure) ou l'excuse.

La couleur de la carte est définie par une énumération Color, l'atout est considéré comme une couleur. Il suffit ensuite d'utiliser l'opérateur de comparaison de carte, pour savoir si une carte est supérieur à une autre, ou non. Pour savoir si une carte est un atout, nous pouvons demander la couleur de la carte, et regarder si c'est de l'atout.

L'énumération Color est définie dans cette classe, car une couleur est attachée à une carte.

Et maintenant, comment donner plusieurs cartes à un joueur ? Deux possibilités se sont offertes à moi : créer un vecteur de pointeurs de cartes dans chaque joueur, ou créer une classe modélisant un paquet de cartes (Deck). D'après ma représentation UML, il est évident que j'ai choisi la deuxième.

5 Deck

Pourquoi un paquet de carte ? Cette question est légitime, pourquoi créer une classe, alors qu'un simple vecteur aurait suffi. La raison est qu'en faisant cela, il est possible de faire toute sorte d'opérations sur un paquet de carte. Evidemment, le joueur aurait pu faire cela aussi, mais un paquet de cartes pourra ainsi composé d'autres classes que juste un joueur.

Description Un paquet de cartes est composé d'une vector de Card *, d'un tableau contenant les nombres de chaque couleurs de cartes, pratique pour vérifier si les joueurs peuvent encore jouer la couleur en jeu, et ne pas

tricher, ainsi que le nombre de têtes et le nombre d'atouts.

Beaucoup d'opération sont possible :

- Enlever une carte à la fin du paquet,
- Ou à un endroit donnée,
- Ajouter une carte à la fin du paquet,
- Générer un paquet de 78 cartes pour jouer au tarot,
- Trier un paquet,
- Ou le mélanger.

Il est également possible de savoir la taille du paquet, le nombre de cartes par couleurs, le nombres d'atout, de têtes, s'il est vide... en bref, l'état du paquet à chaque instant.

Le tarot étant un jeu, il nous faut des joueurs (Player). Qui l'eut-crû ?

6 Player

6.1 Classe abstraite

La classe Player est une classe abstraite. Un jeu de tarot comprend 4 personnes, or un utilisateur d'ordinateur est souvent seul. Il est donc assez logique de créer deux types de joueurs : les joueurs humains (HumanPlayer) et les joueurs contrôler par l'ordinateur (IAPlayer).

Description Je vais quand même détailler cette classe, car beaucoup d'actions sont communes aux deux types de joueurs, ainsi que les attributs complètement communs.

Tout d'abord, chaque joueur possède une main (hand) de type Deck, ainsi que tas (heap) dans lequel le joueur mettra les cartes qu'il a gagné. Il a également un nombre de points et un nom.

Voici les actions communes aux joueurs :

- Rajouter une carte dans la main,
- Pareil, mais dans le tas,
- Trier la main,
- Compter les points à partir du tas,
- Choisir une carte de petite valeur pour en remplacer une autre.

Et évidemment, nous pouvons connaître le nom du joueur, son nombre de points, ainsi que le nombre de bouts qu'il a dans son tas.

Une énumération TypeRound est déclarée dans cette classe, afin de définir de façon claire les types de contrat possible, et que c'est dernier sont rattachés aux joueurs. Je n'ai pas permis le choix de la Garde Contre car cela m'embêtait de rajouter encore du code pour distribuer deux cartes à chaque défenseurs dans ce cas-là.

Classes dérivées La classe Player contient 3 méthodes abstraites pures :

- Choisir un contrat,
- Faire le chien,
- Jouer une carte.

Chacune des deux classes héritées de Player implémente de façon différentes ces trois méthodes, sinon il n’y aurait pas lieu de les déclarer abstraite. Voyons voir comment...

6.2 HumanPlayer

Il n’y a pas grand-chose à dire sur les joueurs humains, car les méthodes proposent juste des interactions aux joueurs via la console.

Seules quelques vérifications sont faites, par exemple que le type de contrat choisi est possible, que le choix de la carte est cohérent avec le jeu, que la couleur de celle-ci suive bien la couleur en cours... en bref, toute tricherie possible du joueur, ainsi que quelques vidages de tampon mémoire, dans le cas où l’utilisateur écrive n’importe quoi.

6.3 IAPlayer

Les joueurs contrôlés par ordinateur sont un peu plus détaillables.

Choisir un contrat Le joueur regarde le nombre de bouts, de têtes et d’atouts. Un bout étant de plus grande importance qu’une tête dans le choix d’un contrat, j’ai décidé d’appliquer un multiplicateur x3 au nombre de bouts. Les atouts sont assez décisifs également, un peu moins que les bouts, j’ai donc appliqué un multiplicateur x2 au nombre d’atouts et aucun multiplicateur au nombre de têtes. Après cela, le joueur additionne ces termes ainsi multipliés, et en fonction de la valeur obtenue, il choisit un type de contrat.

Faire le chien Le joueur ne s’embête pas, il met tout le chien directement dans son tas.

Joueur une carte Le joueur regarde d’abord s’il a encore une carte (pareillement au joueur humain), il renvoie 0 s’il n’en a pas. Sinon, il regarde s’il lui reste des cartes dans la couleur jouée, ou en atout si cette dernière lui fait défaut. Ensuite, il choisit une carte au hasard dans la couleurs demandée. Dans le cas où le joueur n’a aucune carte jouable ou qu’il est l’engageur, il joue n’importe quelle carte. C’est ici que la couleur NoColor est utile, car elle permet de savoir si aucune couleur est jouée, et donc que le joueur est engageur.

Le joueur renvoie ensuite le pointeur de la carte qu’il joue.

7 Game

Cette classe est la classe maîtresse du jeu (d'où son nom), c'est le contrôleur de toute ce qui se passe.

Description Elle a comme attribut un paquet de carte de base, celui qui sera distribué, dans lequel les joueurs remettront leur carte à la fin de chaque manche, un vector de Player *, pour avoir accès aux joueur de la partie, un chien (dog) qui sera proposé au preneur suivant son contrat, le tapis (playMat), ainsi que les indices dans le vector du joueur preneur, du premier joueur distribué pour savoir qui sera le prochain au tour suivant, et celle de l'engageur à chaque nouveau tour de jeu.

Les méthodes sont :

- Exécution de l'ensemble : cette fonction sert à demander au joueur humain s'il souhaite faire une nouvelle partie,
- Nouvelle manche : celle-ci applique toutes les règles d'une manche comme expliquées plus haut,
- Nouveau tour : un tour de tapis pour que chaque joueur pose sa carte, l'excuse est remplacée à la fin si jouée.

Quatrième partie

Conclusion

Il est assez évident que ce programme ne gère pas toutes les règles du tarot, cela aurait été trop compliqué, et cela aurait entraîné une surcharge (pas de double sens ici) de méthodes dans les classes.

J'ai implémenté les aspects les plus importants à mon goût, permettant également une meilleure lisibilité du code, du moins je l'espère.